Implementation Runbook

Microsoft Fabric · dbt · Azure Data Factory
Merchant Sales & Voucher Intelligence — BI Developer Second-Round Practical Task

Scope	Workspace provisioning through to a scheduled, quality-gated daily refresh feeding a Direct Lake semantic model
Layers	Landing → Bronze → Silver → Gold (Kimball star) → Power BI
Artefacts	1 Lakehouse · 1 Warehouse · 4 notebooks · 1 ADF pipeline · 1 schedule trigger · 18 dbt models · 1 SCD2 snapshot · 132 dbt tests
Cadence	Daily 02:00 South Africa Standard Time
Quality gate	dbt test runs BETWEEN silver and gold. Gold is not rebuilt unless every test passes.

Part 0 — The journey of one row through Microsoft Fabric

Before the setup steps, this is the story the architecture tells: what actually happens to a single sales record between the source file landing and an executive seeing it on a page. Every later section exists to serve one of these seven stages.

1	It lands	OneLake Files / ADF Copy	At 02:00 SAST the pipeline copies MerchantSales.csv into LH_MerchantVoucher/Files/landing. Our row is one line of text: `2026-07-31,M002,Umhlanga Value Mart,Free State,Retail,Airtime,2191.80,46`	Nothing is validated yet. The only question asked is 'did the file arrive and is it big enough' — the minRows floor of 20,000 catches a truncated feed here, before it can look like a bad trading day.
2	It is landed raw (BRONZE)	Lakehouse Delta table	Written to bronze_merchant_sales exactly as received — every column still a string. Three columns are added and nothing else changes: _source_file, _ingested_at, _batch_id.	Bronze deliberately applies no business logic. If a number is later disputed, this is the layer that proves what the source actually sent, unmodified. The batch id is what makes 'which load produced this figure?' answerable during an incident.
3	It is cleaned and conformed (SILVER)	Fabric notebook 01_bronze_to_silver	The row is typed (date, decimal, int), trimmed, and de-duplicated at its declared grain of Date × Merchant × VoucherType. Merchant, Region and Channel are DROPPED.	Dropping them is the important move. They also live on MerchantReference, and profiling proved 100% agreement — so keeping both would let two competing versions of 'Region' exist in one model, which is how a report ends up with two different regional totals on two different pages.

4	It is judged against the rules	dbt test — the quality gate	132 tests run against silver. Is the grain still unique? Does every merchant id resolve? Is the date table contiguous? Does total revenue still tie to bronze to the cent?	This gate sits BEFORE gold, not after. If it fails the pipeline stops and gold is left untouched, so the report keeps yesterday's correct numbers instead of publishing today's wrong ones. Our row only continues because every test passed.
5	It joins the star schema (GOLD)	dbt build → Warehouse	The row becomes a fact: date 2026-07-31 becomes date_key 20260731; M002 becomes a merchant surrogate key; Airtime becomes a voucher_type key. Only keys and measures remain — 2191.80 and 46.	Alongside it, snap_merchant checks whether M002's status, region, manager or target changed since yesterday. If they did, a new version row is written and mart_merchant_change_alerts raises it — so a merchant quietly moving to 'At Risk' cannot pass unnoticed.
6	It becomes a number someone reads	Power BI Direct Lake	No import, no copy. The semantic model reads the Delta files directly; the pipeline's last step reframes it. [Total Sales] = SUM(sales_value) now includes our R2,191.80.	The row also lands in mart_merchant_scorecard, where M002's July collapse of –42.5% against its own prior three months drops its health score to 17.3 and puts R571,518 of annualised revenue at risk on the focus list.
7	It is explained	ML notebook + narrative measures	The Isolation Forest scores M002's July as the single most anomalous merchant-month in the dataset, with the reason attached in plain English: sales –38% vs own history, tickets +594%.	An account manager opens the report at 07:00 and sees not a number but a sentence: Umhlanga Value Mart has declined sharply, support volume has risen 693%, and the annualised shortfall is R571,518. That is the point of all six preceding stages.

What the journey is designed to prevent

Each stage exists to stop a specific, common failure. Stage 1 stops a truncated file looking like weak trading. Stage 2 stops a disputed number being unanswerable. Stage 3 stops two versions of the truth entering the model. Stage 4 stops wrong figures reaching an executive. Stage 5 stops a silent master-data change invalidating a comparison. Stage 6 stops a stale copy being reported as live. Stage 7 stops a correct number going unnoticed because nobody knew what it meant.

Part 1 — Microsoft Fabric workspace setup

Everything in this part is done once, per environment. Three environments are provisioned (Dev, Test, Prod) so the deployment pipeline in Part 4 has somewhere to promote to.

1.1	Create the capacity	Fabric admin	Azure portal → Create resource → Microsoft Fabric Capacity → F2 or higher, region South Africa North	Capacity appears in the Fabric admin portal and is Active
1.2	Create three workspaces	Fabric admin	Fabric → Workspaces → New workspace: WS_MerchantVoucher_Dev / _Test / _Prod. Assign the capacity under Advanced → License mode → Fabric capacity	All three workspaces show the capacity name, not 'Pro'
1.3	Create the Lakehouse	Data engineer	In each workspace: New → Lakehouse → LH_MerchantVoucher	Lakehouse opens with empty Tables and Files sections
1.4	Create the landing folder	Data engineer	LH_MerchantVoucher → Files → New subfolder → landing	Files/landing exists and is writable
1.5	Create the Warehouse	Data engineer	New → Warehouse → WH_MerchantVoucher. Run sql/01_create_warehouse.sql	gold schema contains 7 dimensions, 4 facts, 2 marts, 2 ML tables
1.6	Register the service principal	Fabric admin	Entra ID → App registrations → New. Grant the SP Contributor on all three workspaces	SP can authenticate to the Warehouse SQL endpoint
1.7	Capture connection details	Data engineer	Warehouse → Settings → SQL connection string. Store as FABRIC_SQL_ENDPOINT	Endpoint resolves and the SP can connect

Why a Lakehouse AND a Warehouse

The Lakehouse holds bronze and silver as Delta — schema-on-read, cheap, and Spark-native for the notebooks. The Warehouse holds gold, because it gives a T-SQL surface for dbt-fabric and for analysts who will never write PySpark. Both sit on OneLake, so gold reads silver with no data movement.

Part 2 — dbt project

The dbt project runs against a local analytical engine for development (seconds per run, no cloud cost) and against the Fabric Warehouse in production. Identical SQL — only the adapter changes, which is the main reason dbt is in this stack at all.

2.1	Install dbt and the adapters	BI developer	pip install dbt-core dbt-fabric dbt-duckdb	dbt --version lists both adapters
2.2	Set environment variables	BI developer	FABRIC_SQL_ENDPOINT, FABRIC_WAREHOUSE, AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET	dbt debug --target fabric returns All checks passed
2.3	Install package dependencies	BI developer	dbt deps	dbt_packages/ contains dbt_utils, dbt_expectations, codegen
2.4	Load the seeds	BI developer	dbt seed	4 seeds loaded: voucher_type, ticket_type, priority, ticket_status reference
2.5	Build everything in DAG order	BI developer	dbt build	153 pass, 0 warn, 0 error. This runs seeds, the snapshot, 18 models and 132 tests in dependency order
2.6	Verify the SCD2 snapshot behaves	BI developer	python scripts/_test_scd2.py	7/7 assertions pass and the harness restores original state
2.7	Generate and publish documentation	BI developer	dbt docs generate && dbt docs serve	Lineage graph renders; every model shows a description and its tests
2.8	Reconcile against the reference implementation	BI developer	python scripts/05_reconcile.py	27 pass, 1 documented rounding warning, 0 fail

Model layers

	staging (4 views)	silver	Type, trim, deduplicate at the declared grain, apply integrity rules	Descriptive attributes dropped — they belong to the dimension
	intermediate (2 ephemeral)	—	int_merchant_monthly (merchant × month spine, zero-filled) and int_merchant_momentum	Inlined as CTEs; no storage, one definition of 'latest vs prior three'
	snapshots (1)	SCD2	snap_merchant — check strategy on region, channel, active_status, account_manager, target	Source has no reliable last-modified column, so timestamp strategy is unsafe
	marts / core (11)	gold	7 conformed dimensions, 4 facts, plus the Type 2 dimension	Facts at three grains; targets kept in their own month-grain fact
	marts / analytics (2)	gold	mart_merchant_scorecard and mart_merchant_change_alerts	Health Score computed in SQL so Power BI, Excel and ML all inherit one definition

The snapshot is wired to alerting, not just history

mart_merchant_change_alerts diffs consecutive snapshot versions and emits one row per changed field with a severity. A merchant moving to 'At Risk' is Critical; a re-based sales target is High, because it silently invalidates every period-over-period attainment comparison unless someone is told. Capturing history nobody reads is not worth the storage — this is what makes the snapshot earn its place.

Part 3 — Azure Data Factory / Fabric pipeline orchestration

PL_MerchantVoucher_Master.json. Three design decisions carry most of the value and are called out after the steps.

3.1	Create the pipeline	Data engineer	Fabric workspace → New → Data pipeline → PL_MerchantVoucher_Master. Import datafactory/PL_MerchantVoucher_Master.json	Pipeline canvas shows 6 top-level activities
3.2	Create the linked service	Data engineer	Manage → Linked services → New → Fabric Warehouse → LS_Fabric_Warehouse, authenticate with the service principal	Test connection succeeds
3.3	Set the batch ID	(pipeline)	SetVariable: BatchId = @formatDateTime(utcnow(),'yyyyMMddTHHmmssZ')	Every downstream row carries this batch id for lineage
3.4	Log the run start	(pipeline)	Script activity → INSERT INTO audit.pipeline_run (... status 'RUNNING')	A row appears in audit.pipeline_run
3.5	Ingest, metadata-driven	(pipeline)	ForEach over the SourceFiles parameter array (isSequential false, batchCount 4) → Copy activity → Lakehouse bronze table	4 bronze Delta tables written; retry 3 on transient faults
3.6	Validate row counts	(pipeline)	IfCondition per iteration: fail when rowsCopied < item().minRows	A truncated file fails loudly instead of silently producing a zero dashboard
3.7	Transform bronze to silver	(pipeline)	TridentNotebook → notebooks/01_bronze_to_silver.py, parameters batch_id and run_id	4 silver tables written, Z-ORDERed on merchant + date
3.8	Run the data-quality gate	(pipeline)	TridentNotebook → dbt test. Retry 0 — a failing test is a real defect	Notebook exits PASS or FAIL
3.9	Branch on the gate	(pipeline)	IfCondition on exitValue = 'PASS'. False branch → Teams webhook → Fail activity	On failure gold is NOT rebuilt and the previous good load stays live
3.10	Build gold	(pipeline)	TridentNotebook → dbt build --select marts	Dimensions, facts and marts rebuilt in dependency order

3.11	Score the ML models	(pipeline)	TridentNotebook → notebooks/03_ml_anomaly_and_forecast.py	ml_anomaly_scores and ml_sales_forecast written back to gold
3.12	Frame the semantic model	(pipeline)	Web activity → POST /v1.0/myorg/groups/{ws}/datasets/{id}/refreshes, MSI auth	Direct Lake model reframed; last refresh timestamp updates
3.13	Schedule it	Data engineer	Import TR_Daily_0200_SAST.json. Timezone named explicitly, not computed from UTC	Trigger shows Started; next run 02:00 SAST

Part 3b — why the pipeline is shaped this way

1. The data-quality gate sits BETWEEN silver and gold

This is the most important decision in the pipeline. If the gate fails, the pipeline stops, alerts, and leaves the previous good gold layer in place — so the report keeps showing yesterday's correct numbers rather than today's wrong ones. Loading gold first and testing afterwards means the report has already published bad figures by the time anyone reads the alert. Stale-but-correct beats fresh-but-wrong: an executive working from a day-old number makes a slightly late decision; one working from a wrong number makes a wrong decision.

2. Row-count validation catches the failure that does not throw

A truncated or empty source file raises no error. The copy succeeds, the transformation succeeds, and the dashboard shows zero — which looks exactly like a genuinely bad trading day. Per-source minimum row counts (MerchantSales 20,000 · VoucherRedemptions 100,000 · SupportTickets 1,000 · MerchantReference 20) turn that silent failure into a loud one. Thresholds sit well below current volumes so normal fluctuation does not trip them, but far above zero so truncation cannot pass.

3. Metadata-driven ingest, not four hard-coded copy activities

A SourceFiles parameter array carries one object per source and a ForEach iterates it. Adding a fifth source is a parameter change, not a pipeline edit — which matters because pipeline edits require deployment and regression testing while parameter changes do not. It also guarantees all four copies behave identically, so a fix to retry behaviour applies everywhere rather than to whichever activity someone remembered.

Retry policy

	Copy to Bronze	3 retries	60s interval	Transient storage and network faults are common and genuinely self-healing
	Transform to Silver	1 retry	120s interval	Largely deterministic; one retry covers a capacity blip
	Data Quality Gate	0 retries	—	A failing test is a real defect. Retrying delays the alert and risks a flaky pass
	Build Gold	1 retry	120s interval	As above
	ML Scoring	1 retry	180s interval	Longer running, more exposed to capacity contention
	Refresh Semantic Model	2 retries	60s interval	The Power BI REST API returns transient 429s under load

Part 3c — Gold layer ERD

Generated from the dbt manifest, not drawn. **dbt docs shows the DAG** — which model builds from which — and that is build lineage, not entity relationships. It cannot show that fct_support_tickets.priority_key joins to dim_priority, because the fact is built from staging, not from the dimension. The real foreign keys live in the schema tests: every **relationships** test is an enforced statement that a column must resolve to another column. This diagram is derived from those tests, so it cannot drift — drop a join and the test disappears, and so does the line.

	dim_merchant_history	merchant_key	dim_merchant.merchant_key	Enforced by a dbt relationships test
	fct_merchant_sales	date_key	dim_date.date_key	Enforced by a dbt relationships test
	fct_merchant_sales	merchant_key	dim_merchant.merchant_key	Enforced by a dbt relationships test
	fct_merchant_sales	voucher_type_key	dim_voucher_type.voucher_type_key	Enforced by a dbt relationships test
	fct_merchant_target	date_key	dim_date.date_key	Enforced by a dbt relationships test
	fct_merchant_target	merchant_key	dim_merchant.merchant_key	Enforced by a dbt relationships test
	fct_support_tickets	date_key	dim_date.date_key	Enforced by a dbt relationships test
	fct_support_tickets	merchant_key	dim_merchant.merchant_key	Enforced by a dbt relationships test
	fct_support_tickets	priority_key	dim_priority.priority_key	Enforced by a dbt relationships test
	fct_support_tickets	status_key	dim_ticket_status.status_key	Enforced by a dbt relationships test
	fct_support_tickets	ticket_type_key	dim_ticket_type.ticket_type_key	Enforced by a dbt relationships test
	fct_voucher_redemptions	merchant_key	dim_merchant.merchant_key	Enforced by a dbt relationships test
	fct_voucher_redemptions	redeemed_date_key	dim_date.date_key	Enforced by a dbt relationships test
	fct_voucher_redemptions	sold_date_key	dim_date.date_key	Enforced by a dbt relationships test
	mart_merchant_scorecard	merchant_key	dim_merchant.merchant_key	Enforced by a dbt relationships test

Why fourteen tables when the README suggests five

The supplied README suggests a 5-table model. This solution has 14. Five come straight from the README; four exist because the brief's own report requirements (voucher type performance, ticket priority, SLA risk, backlog) need somewhere to put those attributes; one is a grain necessity; and four are extensions I added on judgement and have labelled as such. Presenting all 14 as though the brief demanded them would be overselling it — the honest split is a 10-table core plus 4 extensions.

	Supplied CSV	4	dim_merchant, fct_merchant_sales, fct_support_tickets, fct_voucher_redemptions	Named in the README and supplied as a file
	README-required	1	dim_date	Named in the README, no file — built here
	Brief deliverable	4	dim_priority, dim_ticket_status, dim_ticket_type, dim_voucher_type	Required by a stated deliverable
	Grain necessity	1	fct_merchant_target	Separate grain — additivity
	My extension	4	dim_merchant_history, mart_merchant_change_alerts, mart_merchant_scorecard, snap_merchant	Beyond the brief, added on judgement

The fair criticism, and the answer

The fair criticism is that four dimensions of 4-6 rows each is over-normalisation, and some modellers would keep those as degenerate columns on the ticket fact. That was in fact the first design here — all three ticket dimensions unioned into one table behind a discriminator — and it was worse for two concrete reasons. The three foreign keys on fct_support_tickets could not be relationship-tested, because a test cannot distinguish 'priority_key resolves to a priority' from 'resolves to something, somewhere'. And Power BI cannot build three independent filter paths off one physical table without role-playing copies. Splitting them added 9 enforced foreign keys that previously could not exist.

Part 3d — Reconciliation and financial controls

Controls that PASS matter as much as controls that fail: a control nobody can see is a control nobody trusts. **mart_reconciliation** materialises each check with its expected value, actual value, variance and a derived status.

1	Source control — sales value survives bronze to gold	PASS	R65,521,298.75 both sides	Every step between landing and the star schema is a cast, a filter on invalid rows or a regroup at the same grain. None may change revenue.
2	Source control — voucher rows survive	PASS	120,969 both sides	One row per voucher in, one row out. Any loss is a defect.
3	Source control — ticket rows survive	PASS	1,363 both sides	As above for the ticket fact.
4	Voucher control — redeemed + outstanding = issued	PASS	R22,019,852.75	A voucher is either redeemed or outstanding. If these do not sum to value issued, value has been created or destroyed in the model.
5	Mart control — scorecard ties to the sales fact	PASS	R65,521,298.75	The scorecard is rebuilt through two ephemeral intermediate models; a refactor there is exactly what could silently drop a merchant.
6	Population control — the two value facts do NOT tie	EXPECTED	R43,501,446 variance	The most important control here. MerchantSales is a daily aggregate of ALL transactions (510,127); VoucherRedemptions is a voucher-level extract covering roughly 1 in 4.2 of them (120,969). They describe different populations and must never be forced to tie. Recording that as an EXPECTED variance is what stops someone escalating a R43.5m 'break' — or dividing one by the other and reporting a value-based redemption rate against the wrong denominator.

Fraud and risk controls — what the data supports

	Reversal ticket rate	SUPPORTED	reversal_per_1k_txn	Reversal Query tickets per 1k transactions. Reversals are where value moves back.

	Redemption velocity	SUPPORTED	same_day_redemption_rate	Same-day redemption share. Legitimate for airtime, unusual in bulk.
	Outstanding liability concentration	SUPPORTED	outstanding_value	Unredeemed value sitting with one merchant.
	Behavioural anomaly	SUPPORTED	Isolation Forest	Multivariate shift against a merchant's own history.
	Voucher value outliers	TESTED — NIL	value_outlier_vouchers	Implemented and evaluated: ZERO vouchers beyond 3 SD within type, because the synthetic generator used a bounded distribution. The control runs and correctly returns zero rather than being quietly omitted.
	Duplicate redemption	NOT SUPPORTED	—	voucher_id is unique by construction in the extract, so a duplicate cannot appear. Needs a raw redemption event log with repeated attempts.
	Geographic anomaly	NOT SUPPORTED	—	Needs merchant location, transaction location and timestamp. Only static province at merchant level is supplied.
	PIN / invalid voucher use	NOT SUPPORTED	—	Needs voucher PIN, attempt outcome and failure reason.
	Customer velocity / customer CLV / churn	NOT SUPPORTED	—	There is no customer identifier anywhere in the four source files. Merchant-level lifetime value and attrition risk are built instead, in mart_merchant_value_risk.

Part 4 — Promotion, monitoring and daily operation

4.1	Create the deployment pipeline	Fabric admin	Fabric → Deployment pipelines → New → assign Dev / Test / Prod workspaces	Three stages show item counts
4.2	Parameterise per stage	Data engineer	Deployment rules: workspace id, semantic model id, notebook ids. Never literals	The same pipeline JSON promotes unchanged
4.3	Deploy Dev → Test	BI developer	Compare → Deploy. Run the pipeline once end to end in Test	audit.pipeline_run shows SUCCEEDED and dbt tests pass in Test
4.4	Deploy Test → Prod	Release manager	Compare → Deploy, then enable TR_Daily_0200_SAST	Trigger Started; first scheduled run completes
4.5	Configure failure alerting	Data engineer	Teams webhook on the DQ-gate false branch; Fabric Monitor → set alert on pipeline failure	A deliberately failed run posts to the channel
4.6	Configure change alerting	BI developer	Surface mart_merchant_change_alerts on the Insights page; optionally add a Data Activator rule on severity = 'Critical'	A merchant moving to 'At Risk' raises an alert within one refresh cycle
4.7	Monitor source freshness	Data engineer	dbt source freshness — warn at 26h, error at 48h	A stalled upstream feed surfaces as a dbt error, not as stale numbers in the report
4.8	Review the run daily	BI support	Check audit.pipeline_run, then the dbt test summary, then the report refresh timestamp	Three green signals before 07:00

Daily operational checklist

1	Pipeline completed	audit.pipeline_run	status = 'SUCCEEDED' for today's batch	If RUNNING past 03:00, check Fabric Monitor for a stuck notebook

2	Quality gate passed	dbt test output	153 pass, 0 error	If failed, gold was intentionally not rebuilt — fix the source, do not force the run
3	Row counts sane	bronze tables	Within the configured minRows floors	A large drop with a passing pipeline still warrants a look at the source file
4	Semantic model framed	Power BI dataset	Last refresh within the last 6 hours	Direct Lake still needs framing after the Delta tables change
5	Change alerts triaged	mart_merchant_change_alerts	No unreviewed Critical rows	An 'At Risk' flag should be cross-checked against the computed health score before anyone acts on it

Known issue to raise with the business on day one

The supplied BaseMonthlySalesTarget sits roughly 6.1x below realised sales for all 25 merchants, so raw target attainment reads about 614%. The consistency across every merchant points to a basis or units error rather than genuine outperformance. The supplied value is retained unchanged for transparency and the report uses a relative Target Attainment Index instead. Confirming the intended basis is the first question for Finance.